SPPU OPERATING SYSTEMS LAB

1. Address Book Using Shell Script

AIM

To develop a menu-driven address book in Bash shell scripting to create, display, insert, delete, and modify records.

THEORY

A shell is a command interpreter that executes user commands. A shell script is a text file containing commands executed by a shell such as Bash. The address book stores contact details (name, phone number, and address) in a text file, with fields separated by a delimiter. A menu is repeated using a loop, and a case statement selects the required operation. The mkdir and touch commands create the directory and file; cat displays records; echo with the append operator adds records; grep can filter records for deletion or modification. Input is collected using read. File handling makes the records persistent across script executions. Care must be taken to validate inputs and handle names safely.

SPPU OPERATING SYSTEMS LAB

2. Process Management: fork(), wait(), Zombie and Orphan

AIM

To demonstrate process creation and synchronization using fork() and wait(), parent-child sorting, and zombie and orphan processes in Linux.

THEORY

A process is a program in execution. The fork() system call creates a child process. It returns zero in the child, a positive child PID in the parent, and -1 on failure. The parent and child execute independently and have separate virtual address spaces; sorting an array in one process does not modify the other process's copy. The wait() system call allows a parent to wait for a child to terminate and collect its exit status. A zombie is a terminated child whose exit status has not yet been collected by its parent. An orphan is a running child whose original parent has terminated; Linux reassigns it to an appropriate reaper or subreaper. The getpid() and getppid() calls identify process and parent IDs.

SPPU OPERATING SYSTEMS LAB

3. Producer-Consumer Using Semaphore and Mutex

AIM

To implement the bounded-buffer Producer-Consumer problem using POSIX threads, counting semaphores, and a mutex.

THEORY

The Producer-Consumer problem is a classical synchronization problem involving a shared buffer. The producer inserts items and the consumer removes them. Without synchronization, concurrent access can cause race conditions, buffer overflow, or buffer underflow. A counting semaphore empty records the number of vacant buffer slots and is initialized to the buffer capacity. Another counting semaphore full records occupied slots and is initialized to zero. The producer waits on empty, locks a mutex, inserts an item, unlocks the mutex, and posts to full. The consumer waits on full, locks the mutex, removes an item, unlocks the mutex, and posts to empty. Circular indexing using modulo wraps insertion and removal indices. The mutex ensures mutual exclusion in the critical section, while semaphores coordinate availability.

SPPU OPERATING SYSTEMS LAB

4. Dining Philosophers Using Semaphores

AIM

To implement the Dining Philosophers synchronization problem using threads and semaphores and prevent deadlock.

THEORY

Five philosophers sit around a table with five shared forks. Each philosopher alternates between thinking and eating and must acquire two adjacent forks to eat. If every philosopher holds one fork and waits for another, circular waiting may cause deadlock. The four necessary conditions for deadlock are mutual exclusion, hold and wait, no preemption, and circular wait. In this implementation, each fork is represented by a binary semaphore initialized to one. A room counting semaphore is initialized to N - 1 (four for five philosophers), so at most four philosophers may attempt to acquire forks at once. Each philosopher waits for a room permit, acquires both forks, eats, releases both forks, and returns the room permit. This arrangement prevents deadlock, though it does not guarantee freedom from starvation.

SPPU OPERATING SYSTEMS LAB

5. Banker's Algorithm

AIM

To implement the safety algorithm of Banker's Algorithm in C and determine whether a resource-allocation state is safe.

THEORY

Banker's Algorithm is a deadlock-avoidance technique that evaluates resource allocation before allowing the system to enter an unsafe state. The Allocation matrix gives resources currently assigned to each process, the Max matrix gives maximum declared requirements, and the Available vector gives free resources. The Need matrix is calculated as Need = Max - Allocation. Initially, all processes are unfinished and the working resource vector equals Available. An unfinished process may be selected when its Need is less than or equal to Work for every resource type. The algorithm then assumes that the process finishes and releases its Allocation back to Work. This continues until all processes finish or no process can proceed. If all finish, the system is safe and the completion order is a safe sequence. An unsafe state does not necessarily mean deadlock has already occurred.

SPPU OPERATING SYSTEMS LAB

6. Full-Duplex IPC Using Two Named FIFOs

AIM

To establish full-duplex communication between two independent Linux processes using two named pipes (FIFOs), count characters, words and lines, and return the result.

THEORY

Inter-Process Communication (IPC) allows separate processes to exchange data. A named pipe, also called a FIFO, is a special file-system object created using mkfifo(). It transfers data in first-in, first-out order. A single FIFO provides a one-way byte stream, so two FIFOs are used for full-duplex communication: one sends text from process 1 to process 2 and the other returns the result. The first process sends sentences. The second process reads the text, counts characters, words, and lines, writes the counts to a result file, and sends the file contents back through the second FIFO. The processes must be started independently, without relying on fork() to create the second process. Proper open/read/write ordering and end-of-message handling are important to avoid blocking and incomplete data.

SPPU OPERATING SYSTEMS LAB

7. Parent-Child IPC Using Two Unnamed Pipes

AIM

To implement full-duplex communication between a parent and child process using two unnamed pipes, sending a filename and returning its contents.

THEORY

An unnamed pipe is a kernel-managed byte stream created using pipe(). It has a read end and a write end and is commonly used between related processes. The fork() call creates a child that inherits the pipe file descriptors. Because each pipe transfers data in one direction, two pipes are used for full-duplex communication. The parent sends a pathname to the child through the first pipe. The child reads the pathname, opens the requested file, and sends its contents back through the second pipe. Each process closes unused pipe ends to prevent descriptor leaks and to allow correct end-of-file detection. The read() and write() system calls transfer bytes; their return values must be checked because a single call may transfer fewer bytes than requested. The parent uses wait() to collect the child's termination status.

SPPU OPERATING SYSTEMS LAB

8. Disk Scheduling: SSTF, SCAN and C-LOOK

AIM

To implement SSTF, SCAN, and C-LOOK disk scheduling algorithms and calculate total disk-head movement for a given request queue.

THEORY

Disk scheduling decides the order in which pending disk-cylinder requests are serviced to reduce head movement and seek time. SSTF (Shortest Seek Time First) selects the pending request closest to the current head position; it can cause starvation of distant requests. SCAN moves the head in one direction, servicing requests along the way, continues to the disk end, and then reverses direction to service remaining requests. C-LOOK services requests in one direction only up to the last request in that direction, then jumps to the lowest pending request and continues upward; it does not travel to the physical disk boundary unnecessarily. For this assignment, the initial direction is toward higher-numbered cylinders. Total head movement is the sum of absolute differences between successive head positions, including the C-LOOK wraparound jump. Requests must be serviced exactly once.